Which Speedrun Tricks Help on Their Own? An Ablation of RoPE, RMSNorm and Muon in nanoGPT

Shahbaz Singh
Jodrey School of Computer Science, Acadia University
Draft for COMP 5923 check-in 3 — October 28, 2026

Abstract. *To be written once results are in (check-in 5).*

1 Introduction

Speedrun projects such as modded-nanoGPT [1] train a small GPT-style model to a fixed validation loss as fast as possible. They do this by stacking many changes on top of Karpathy's nanoGPT [2], including rotary position embeddings (RoPE) [3], RMSNorm [4] and the Muon optimizer [5]. Because the changes are added cumulatively, each one is measured against a baseline that already includes all the earlier ones. That makes it hard to tell how much a single change helps by itself, and whether it would help a plain model at all. Narang et al. [6] found that most proposed transformer modifications did not give meaningful improvements when they were re-implemented in one shared codebase. This is a reason not to assume that every component of a speedrun stack is useful on its own.

This project asks:

> *Among RoPE, RMSNorm, and the Muon optimizer, which individually reduce validation loss relative to vanilla nanoGPT when trained at matched compute on Shakespeare-character data, and do any effects hold at small GPT-2 scale?*

The hypothesis is that the three changes do not contribute equally. At matched compute, at least one of them will not reduce validation loss by a statistically significant amount compared with the baseline. Based on the original RMSNorm paper [4], which reports accuracy similar to LayerNorm, RMSNorm is expected to make little or no difference to the loss. Muon is expected to show the largest effect at a matched number of steps, but a smaller one at matched wall-clock time, because each Muon step does extra work.

2 Related work

To be expanded from my annotated bibliography (check-in 2): modded-nanoGPT [1], RoPE [3], RMSNorm [4], Muon [5], Narang et al. [6] and Bouthillier et al. [7].

3 Methods

3.1 Study design

The study is a controlled, one-factor-at-a-time comparison. There are four conditions: the nanoGPT baseline, and three variants that each change exactly one component of it (Table 1). Every condition uses the same data, model size, batch size, number of training steps, learning-rate schedule and evaluation procedure, so a difference in loss can be attributed to the one component that changed. Each condition is trained with several random seeds, and each variant is compared with the baseline across seeds, never as a single pair of runs. The three variants are deliberately not combined: the question is whether each change helps on its own, not whether the full stack helps.

Table 1: The four experimental conditions.

Condition	Difference from the baseline
`baseline`	nanoGPT: learned absolute position embeddings, LayerNorm, AdamW for all parameters
`rope`	Learned position embeddings removed; RoPE applied to queries and keys in every attention layer
`rmsnorm`	Every LayerNorm (two per block and the final one) replaced by RMSNorm
`muon`	The 2D weight matrices inside the transformer blocks are updated by Muon instead of AdamW

3.2 Data

The main experiment uses the tiny Shakespeare corpus [8] at the character level, prepared as in nanoGPT. The text has 1,115,394 characters and a vocabulary of 65 characters. The first 90% (1,003,854 characters) is the training split and the last 10% (111,540 characters) is the validation split. The split is contiguous, so the validation text comes from plays the model has not seen. There is no separate test split. The consequences of this are discussed in the limitations section.

3.3 Baseline model and training

The baseline is nanoGPT's Shakespeare-char configuration (Table 2): a 6-layer, 6-head transformer with 384-dimensional embeddings, a context of 256 characters and dropout of 0.2. It has 10.65M parameters, not counting the position table. The input embedding and the output layer share weights, and the residual output projections use GPT-2's scaled initialization.

Table 2: Baseline hyperparameters (nanoGPT's Shakespeare-char settings).

Setting	Value	Setting	Value
Layers / heads	6 / 6	Optimizer	AdamW, $\beta = (0.9, 0.99)$
Embedding dim.	384	Peak learning rate	10^{-3} (tuned, §3.6)
Context length	256	Weight decay	0.1 (2D weights only)
Dropout	0.2	Warmup	100 steps, linear
Batch size	64 sequences	Decay	cosine to 0.1 × peak
Training steps	5,000	Gradient clipping	1.0 (global norm)
Tokens seen	81.9M (≈ 82 epochs)	Precision	bf16 or fp16 autocast

The model is my own reimplementation in a single configurable codebase, following nanoGPT's `model.py` and `train.py`. One structural difference was needed: the query, key and value projections are three separate linear layers instead of one fused layer. For AdamW this is mathematically identical, because the initialization is the same and AdamW updates each weight independently. It allows Muon to orthogonalize the three projections separately, which is what the Muon reference implementation does.

3.4 The three variants

RoPE. The learned position table is removed, and the query and key vectors in every attention layer are rotated by an angle that depends on their position, as in Su et al. [3] (base 10,000). This removes the 98,304 position-embedding parameters and leaves the rest of the model unchanged.

RMSNorm. Each LayerNorm is replaced by RMSNorm [4], which divides by the root mean square of the input instead of subtracting the mean and dividing by the standard deviation. nanoGPT's LayerNorm has no bias, so both versions have the same parameters, and the only difference is the missing mean subtraction.

Muon. Following the Muon writeup [5], only the weight matrices inside the transformer blocks (Q, K, V, attention output and the two MLP matrices in each layer) are optimized with Muon. The embedding/output matrix and the LayerNorm weights stay on AdamW with the baseline's settings. Muon uses momentum 0.95 with Nesterov, 5 Newton–Schulz steps per update, and the same weight decay and

learning-rate schedule shape as the baseline. Its peak learning rate is tuned separately (§3.6), since Muon updates are on a different scale from AdamW updates.

3.5 Matching compute

RoPE and RMSNorm change the cost of a training step by a negligible amount. For these, compute is matched by giving every run the same number of steps and therefore the same number of training tokens (5,000 steps, 81.9M tokens).

Muon is different: the Newton–Schulz iterations add extra matrix multiplications to every step. Muon is therefore evaluated in two ways:

1. **Matched steps**: 5,000 steps, the same as every other condition.
2. **Matched wall-clock time**: on the same GPU type, I measure the mean training time per step of the baseline t_{base} and of Muon t_{muon}, excluding evaluation. Muon is then trained for $N = \lfloor 5000 \cdot t_{\text{base}}/t_{\text{muon}} \rfloor$ steps, with its learning-rate schedule compressed to end at step N, so that it uses the same training time as the baseline.

The second comparison is closer to what the speedrun actually cares about. Step times depend on the GPU, so both timings have to come from the same GPU type.

3.6 Learning-rate tuning

To avoid giving any condition an advantage through better tuning [6], every condition gets the same small tuning budget. For each condition, I train one run with seed 0 at four peak learning rates and pick the one with the lowest validation loss on the fixed evaluation batches at the end of training. For `baseline`, `rope` and `rmsnorm`, the AdamW learning rate is swept over $\{3, 6, 10, 20\} \times 10^{-4}$. For `muon`, the Muon learning rate is swept over $\{0.005, 0.01, 0.02, 0.04\}$, centred on the reference default of 0.02, while its AdamW parameters keep the baseline's chosen learning rate. Seed 0 is used only for tuning. The main runs use seeds 1–5, so the tuning runs never appear in the comparison.

3.7 Evaluation metrics

The primary metric is the **final validation loss**: the mean cross-entropy, in nats per character, over the whole validation split. It is computed after training on every non-overlapping 256-character window of the split, with dropout off. Every run is therefore scored on exactly the same text, and no random sampling is involved in the final number.

Secondary measurements are:

- **Loss curves.** Every 250 steps, training and validation loss are estimated on 50 fixed batches per split. The same batches are used for every run, so the curves of different runs can be compared point by point. They show whether a change speeds up early training even if the final loss is similar.
- **Training time per step**, used for the matched-time comparison.

3.8 Seeds and statistical analysis

The seed sets both the weight initialization (and dropout masks) and the order of the training batches, the two main sources of randomness identified by Bouthillier et al. [7]. Each condition is trained with 5 seeds (1–5). Each variant is compared with the baseline using Welch's t-test [9], with a Holm correction [10] because there are three comparisons, and I report the mean difference with a 95% confidence interval. With only 5 seeds a small effect may not be significant, so a non-significant result will not be treated as proof that a trick does nothing.

3.9 Confirmation at GPT-2 small scale

If time and budget allow, the variants that show an effect at Shakespeare scale will be re-run on a GPT-2 small model (124M parameters) on a subset of web text, with 2–3 seeds each and a $20–50 budget. With so few runs this would only check whether the direction of the effect holds. This part may be cut (see check-in 3).

3.10 Implementation and verification

All conditions share one codebase. Each run saves its configuration, loss curve, final loss and timing to a file, and a separate script produces the tables and statistics from those files. I also wrote a few tests for the components (for example, that RoPE depends only on relative position and that the model can't see future tokens). So far the pipeline has only been run on CPU with a much smaller model, to check that it works.

4 Results

To be added for check-in 4: learning-rate sweep, main comparison at matched steps (table + loss curves), Muon at matched wall-clock time, GPT-2-small confirmation.

5 Limitations and threats to validity

To be written for check-in 5. Points already noted: tuning and evaluation use the same validation split (no separate test split); small model and a single small dataset, trained for ≈ 82 epochs, so overfitting and dropout interact with each change; only 5 seeds; wall-clock timing depends on the GPU and on whether a fused kernel exists for each component; a single reimplementation, so results may not transfer to other codebases [6]; GPT-2-small runs are short and few.

References

[1] K. Jordan, *modded-nanogpt*. (2024). [Online]. Available: https://github.com/KellerJordan/modded-nanogpt

[2] A. Karpathy, *nanoGPT*. (2022). [Online]. Available: https://github.com/karpathy/nanoGPT

[3] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," *Neurocomputing*, vol. 568, 2024.

[4] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[5] K. Jordan *et al.*, "Muon: An optimizer for hidden layers in neural networks." [Online]. Available: https://kellerjordan.github.io/posts/muon/

[6] S. Narang, H. W. Chung, Y. Tay, W. Fedus, and others, "Do Transformer Modifications Transfer Across Implementations and Applications?," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[7] X. Bouthillier, P. Delaunay, M. Bronzi, and others, "Accounting for Variance in Machine Learning Benchmarks," in *Proceedings of Machine Learning and Systems (MLSys)*, 2021.

[8] A. Karpathy, *char-rnn (tiny Shakespeare dataset)*. (2015). [Online]. Available: https://github.com/karpathy/char-rnn

[9] B. L. Welch, "The Generalization of 'Student's' Problem when Several Different Population Variances are Involved," *Biometrika*, vol. 34, 1947.

[10] S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure," *Scandinavian Journal of Statistics*, vol. 6, 1979.